

```
package com.horizons.ui.aesc
```

```
import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Context
import android.content.Intent
import android.os.IBinder
import android.os.PowerManager
import android.util.Log
import io.ktor.server.application.*
import io.ktor.server.engine.*
import io.ktor.server.netty.*
import io.ktor.server.routing.*
import io.ktor.server.websocket.*
import io.ktor.websocket.*
import kotlinx.coroutines.*
import java.io.*
import java.net.Socket
import java.nio.charset.StandardCharsets
```

```
/**
```

```
 * AescDaemonOrchestrator.kt — Production Android Service for the Æsc Terminal Daemon
 *
```

```
 * Core Responsibilities:
```

```
 * 1. Operates as a foreground START_STICKY service with wake-lock protection to defeat
Android LMK.
 * 2. Maintains persistent local ADB loopback to 127.0.0.1:5555 via Wireless Debugging (UID
2000 shell privileges).
 * 3. Hosts local Ktor Netty WebSocket server on port 8080 (/shell) for frontend communication.
 * 4. Supervises Node.js MCP (Model Context Protocol) child process and restarts on
unexpected exits.
```

```
 */
```

```
class AescDaemonOrchestrator : Service() {
```

```
    companion object {
```

```
        private const val TAG = "AescDaemonOrchestrator"
        private const val NOTIFICATION_CHANNEL_ID = "aesc_daemon_channel"
        private const val NOTIFICATION_ID = 5555
        private const val ADB_HOST = "127.0.0.1"
        private const val ADB_PORT = 5555
        private const val KTOR_PORT = 8080
```

```
}
```

```
private val serviceScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
private var wakeLock: PowerManager.WakeLock? = null
private var ktorServer: NettyApplicationEngine? = null
private var mcpProcess: Process? = null
private var isRunning = false
```

```
override fun onCreate() {
    super.onCreate()
    Log.i(TAG, "Initializing Æsc Daemon Orchestrator...")
    acquireWakeLock()
    startForegroundServiceNotification()
}
```

```
override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    if (!isRunning) {
        isRunning = true
        serviceScope.launch { startAdbLoopbackWatchdog() }
        serviceScope.launch { startKtorWebSocketServer() }
        serviceScope.launch { superviseMcpProcess() }
        Log.i(TAG, "Æsc Daemon Orchestrator active across ADB, Ktor, and MCP pipelines.")
    }
    return START_STICKY
}
```

```
private fun acquireWakeLock() {
    val powerManager = getSystemService(Context.POWER_SERVICE) as PowerManager
    wakeLock = powerManager.newWakeLock(PowerManager.PARTIAL_WAKE_LOCK,
"AescDaemon:WakeLockTag").apply {
        setReferenceCounted(false)
        acquire(24 * 60 * 60 * 1000L) // 24 hours
    }
    Log.i(TAG, "Partial wake-lock acquired.")
}
```

```
private fun startForegroundServiceNotification() {
    val channel = NotificationChannel(
        NOTIFICATION_CHANNEL_ID,
        "Æsc Terminal Daemon",
        NotificationManager.IMPORTANCE_LOW
    ).apply {
        description = "Active background orchestrator holding ADB loopback and WebSocket
```

```

bridges"
    }
    val manager = getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
    manager.createNotificationChannel(channel)

    val notification: Notification = Notification.Builder(this, NOTIFICATION_CHANNEL_ID)
        .setContentTitle("Æsc Terminal Daemon Active")
        .setContentText("Local ADB (5555) & WebSocket (8080) operational")
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setOngoing(true)
        .build()

    startForeground(NOTIFICATION_ID, notification)
}

private suspend fun startAdbLoopbackWatchdog() {
    withContext(Dispatchers.IO) {
        while (isRunning) {
            try {
                val socket = Socket(ADB_HOST, ADB_PORT)
                socket.soTimeout = 3000
                socket.close()
                Log.d(TAG, "ADB loopback healthy on $ADB_HOST:$ADB_PORT")
            } catch (e: Exception) {
                Log.w(TAG, "ADB loopback dropped or unavailable: ${e.message}. Retrying
connect...")
                executeShellCommand("adb connect $ADB_HOST:$ADB_PORT")
            }
            delay(10000)
        }
    }
}

private fun startKtorWebSocketServer() {
    try {
        ktorServer = embeddedServer(Netty, port = KTOR_PORT, host = "0.0.0.0") {
            install(WebSockets)
            routing {
                websocket("/shell") {
                    Log.i(TAG, "Client connected to /shell WebSocket.")
                    for (frame in incoming) {
                        if (frame is Frame.Text) {

```

```

        val receivedCommand = frame.readText()
        val output = executeShellCommand(receivedCommand)
        send(Frame.Text(output))
    }
}

}.start(wait = false)
Log.i(TAG, "Ktor WebSocket server started on ws://0.0.0.0:$KTOR_PORT/shell")
} catch (e: Exception) {
    Log.error(TAG, "Failed to initialize Ktor WebSocket server: ${e.message}")
}
}

private fun executeShellCommand(command: String): String {
    return try {
        val process = Runtime.getRuntime().exec(arrayOf("sh", "-c", command))
        val reader = BufferedReader(InputStreamReader(process.inputStream,
StandardCharsets.UTF_8))
        val output = StringBuilder()
        var line: String?
        while (reader.readLine().also { line = it } != null) {
            output.append(line).append("\n")
        }
        process.waitFor()
        output.toString()
    } catch (e: Exception) {
        "Error executing shell command: ${e.message}"
    }
}

private suspend fun superviseMcpProcess() {
    withContext(Dispatchers.IO) {
        val mcpScriptPath =
"$filesDir.absolutePath/novaexopia/mcp_connectors/launch_mcp_servers.sh"
        while (isRunning) {
            try {
                Log.i(TAG, "Starting MCP server child process...")
                mcpProcess = ProcessBuilder("sh", mcpScriptPath)
                    .redirectErrorStream(true)
                    .start()

                val reader = BufferedReader(InputStreamReader(mcpProcess!!.inputStream))

```

```

        var line: String?
        while (reader.readLine().also { line = it } != null) {
            Log.d("MCP_SUBPROCESS", line ?: "")
        }
        mcpProcess?.waitFor()
        Log.w(TAG, "MCP process terminated. Restarting in 5 seconds...")
    } catch (e: Exception) {
        Log.e(TAG, "MCP supervisor error: ${e.message}")
    }
    delay(5000)
}
}
}

```

```

override fun onDestroy() {
    super.onDestroy()
    isRunning = false
    serviceScope.cancel()
    ktorServer?.stop(1000, 2000)
    mcpProcess?.destroyForcibly()
    wakeLock?.let { if (it.isHeld) it.release() }
    Log.i(TAG, "Æsc Daemon Orchestrator terminated cleanly.")
}

```

```

    override fun onBind(intent: Intent?): IBinder? = null
}
private fun Log.error(tag: String, msg: String) = Log.e(tag, msg)

```